Edge-AI Based Human and Animal Detection using ESP32-CAM

Dhanush Babu Ravinuthala
Matrikel-Nr: 1610103

Industrial AI – Edge AI in Industrial Applications (AI5252)
Winter Semester 2025/26

https://github.com/rdb9/Edge_AI-Human-Animal-Prediction/

---

1. Motivation and Problem Statement

Monitoring environments for the presence of humans or animals has numerous applications in security, wildlife monitoring, and industrial automation. Traditional monitoring systems rely on continuous human observation or centralized cloud-based processing. These approaches introduce challenges such as:

- High latency
- Network dependency
- Privacy concerns
- Increased operational cost

With the advancement of **Edge Artificial Intelligence (Edge-AI)**, intelligent models can now be deployed directly on embedded devices such as microcontrollers and smart cameras.

This project aims to develop a **low-cost Edge-AI vision system using ESP32-CAM** capable of detecting whether a captured image contains a **human or an animal**. The classification is performed directly on the device using a lightweight machine learning model generated through **Edge Impulse**.

---

2. Project Objective

The primary objective of this project is to design and implement a **binary image classification system** running on an embedded device.

The system should:

• Capture an image using ESP32-CAM
• Process the image locally on the device

• Classify the image into one of two classes:

1.human
2.animal

The model should run **entirely on the edge device without cloud processing**, ensuring:

- Low latency
- Reduced network dependency
- Improved privacy
- Efficient resource usage

---

3. System Architecture

The proposed Edge-AI system consists of three major components:

3.1 Edge Device

ESP32-CAM performs:

- Image acquisition
- On-device ML inference
- Result output via serial monitor

3.2 Machine Learning Platform

Edge Impulse Studio is used for:

- Dataset management
- Feature extraction
- Model training
- Model optimization
- Deployment

3.3 Development Environment

Arduino IDE is used to:

- Upload the trained model to ESP32-CAM
- Execute inference code
- Monitor classification results

3.4 System Workflow

The system operates through the following pipeline:

1. ESP32-CAM captures an image.
2. Image is resized to **96×96 resolution**.
3. Preprocessing converts the image into features.
4. The trained neural network performs classification.
5. Prediction results are printed on the serial monitor.

---

4. Hardware Components

The hardware components used in this project include:

Component	Description
ESP32-CAM	Microcontroller with camera module
ESP-CAM-MB	Programming and USB interface module
USB Cable	Connection between device and computer
Laptop	Used for development and model training

ESP32-CAM is a low-cost microcontroller with integrated camera and Wi-Fi capability, making it suitable for embedded computer vision applications.

---

5. Software Tools and Technologies

The project uses the following software tools:

Tool	Purpose
Edge Impulse Studio	Machine learning model development
Arduino IDE	Firmware programming
ESP32 Arduino Core	Board support package
Edge Impulse Arduino Library	On-device ML inference
C++	Embedded programming language

Edge Impulse simplifies the process of creating **TinyML models optimized for embedded devices**.

---

6. Dataset Preparation

The dataset used in this project consists of images belonging to two categories:

1.Human
2.Animal

Images were collected and uploaded directly into **Edge Impulse Studio**.

Dataset preparation steps included:

1. Creating two labels in Edge Impulse
2. Uploading training images
3. Automatically splitting dataset into training and testing sets

The images include variations such as:

- Different lighting conditions
- Different backgrounds
- Various animal species
- Different human poses

This improves the generalization capability of the trained model.

---

7. Model Development using Edge Impulse

The machine learning pipeline was created using the **Impulse Design interface**.

The impulse pipeline consists of three main blocks:

Image Data Block

Input parameters:

- Image width: **96 pixels**
- Image height: **96 pixels**
- Resize mode: **Squash**

These settings reduce computational load while preserving important visual features.

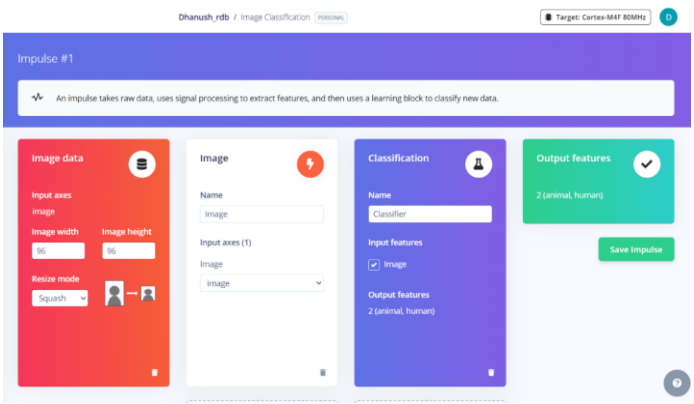


Fig:-1 Impulse

Image Processing Block

The image block extracts visual features from the input image and converts them into a format suitable for the neural network.

Classification Block

A lightweight neural network is used to classify the image into the two classes:

1.human
2.animal

---

8. Feature Generation

After defining the impulse pipeline, the next step involved generating features from the dataset.

Feature generation performs signal processing on input images and produces processed feature vectors that represent the essential information of the image.

The Edge Impulse DSP pipeline ensures efficient feature extraction suitable for embedded inference.

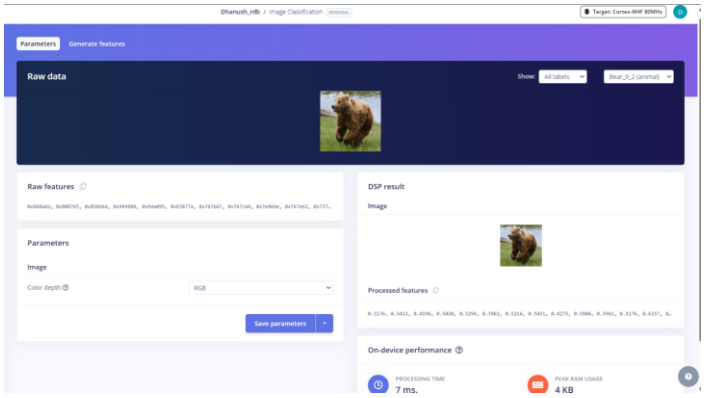


Fig:- 2 Feature generation

---

9. Model Training

The classifier was trained using the neural network training interface provided by Edge Impulse.

Training configuration:

Parameter	Value
Training cycles	10
Learning rate	0.0005
Validation set	20%
Batch size	32

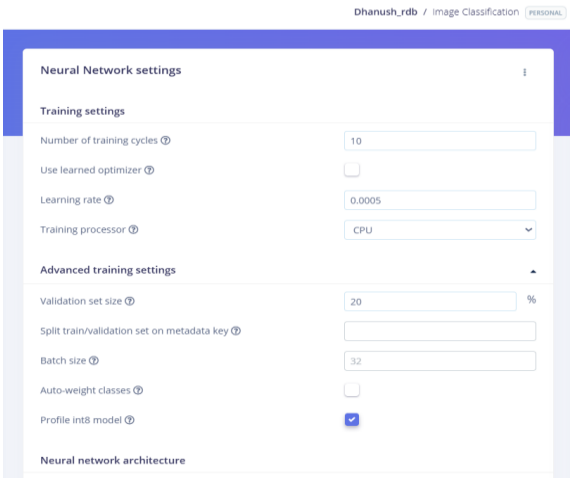


Fig:- 3 Classifier

Model Architecture

The neural network architecture includes:

- Input layer
- Convolution layers
- Pooling layers
- Flatten layer
- Dropout layer
- Output layer

Output layer contains **2 neurons**, representing the two classes.

Training Results

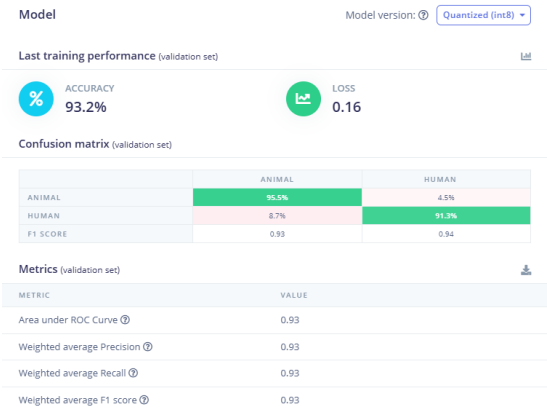
The trained model achieved the following performance:

Accuracy:

93.2 %

Additional metrics:

- Weighted Precision ≈ 0.93
- Weighted Recall ≈ 0.93
- Weighted F1 Score ≈ 0.93



Model

Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY 93.2%

LOSS 0.16

Confusion matrix (validation set)

	ANIMAL	HUMAN
ANIMAL	95.5%	4.5%
HUMAN	8.7%	91.3%
F1 SCORE	0.93	0.94

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	0.93
Weighted average Precision	0.93
Weighted average Recall	0.93
Weighted average F1 score	0.93

Fig:- 4 Confusion Matrix

The confusion matrix shows strong classification performance for both human and animal classes.

---

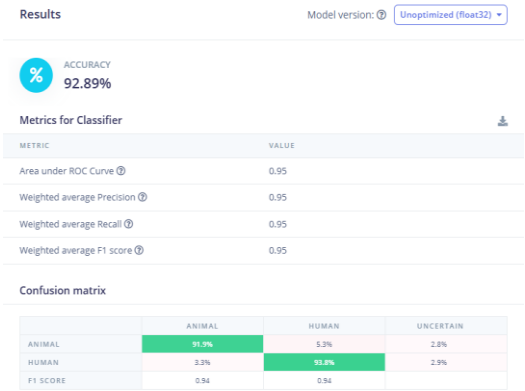
10. Model Testing

The trained model was evaluated using the testing dataset.

The testing stage verifies how well the trained model performs on unseen data.

Model testing results showed:

Accuracy ≈ 92.89 %



Results

Model version: Unoptimized (float32)

ACCURACY 92.89%

Metrics for Classifier

METRIC	VALUE
Area under ROC Curve	0.95
Weighted average Precision	0.95
Weighted average Recall	0.95
Weighted average F1 score	0.95

Confusion matrix

	ANIMAL	HUMAN	UNCERTAIN
ANIMAL	91.9%	5.3%	2.8%
HUMAN	3.3%	93.8%	2.9%
F1 SCORE	0.94	0.94	

Fig:- 5 Classifier Metrics

This demonstrates that the model generalizes well for the classification task.

---

11. Model Deployment

After successful training and testing, the model was deployed using the **Arduino Library deployment option**.

Deployment configuration:

Deployment target: *Arduino Library*
Optimization: *Quantized (int8)*
Inference engine: *EON Compiler*

Model performance estimates:

Metric	Value
Inference time	555 ms
Peak RAM usage	182.8 KB
Flash usage	70.2 KB

Quantization reduces memory usage and improves inference speed on microcontrollers.

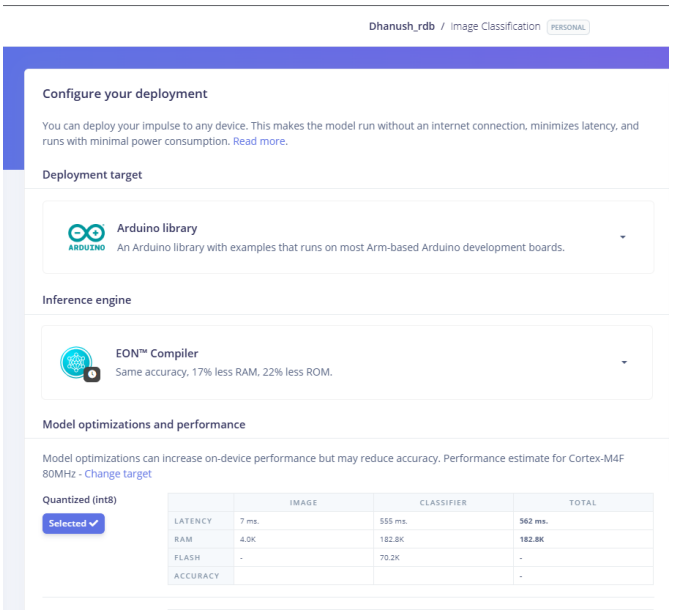


Fig:- 6

12. ESP32-CAM Implementation

The trained model was downloaded as an **Arduino library** and integrated into the ESP32-CAM firmware.

The implementation performs:

1. Camera initialization
2. Image capture
3. Image preprocessing
4. Model inference
5. Result display

A modification was introduced to ensure **static inference**:

The system captures **one image, performs classification, and stops execution**.

Example output:

Prediction Result:
Human: 0.91
Animal: 0.09

This confirms successful deployment of the Edge-AI model on the ESP32 device.

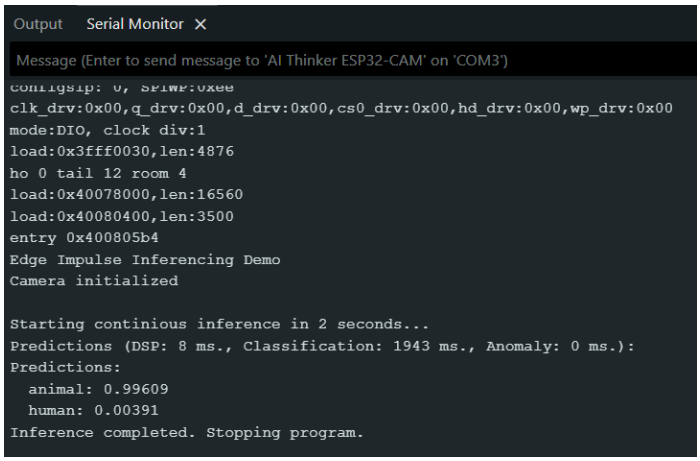


Fig:- 7 Output for an animal image

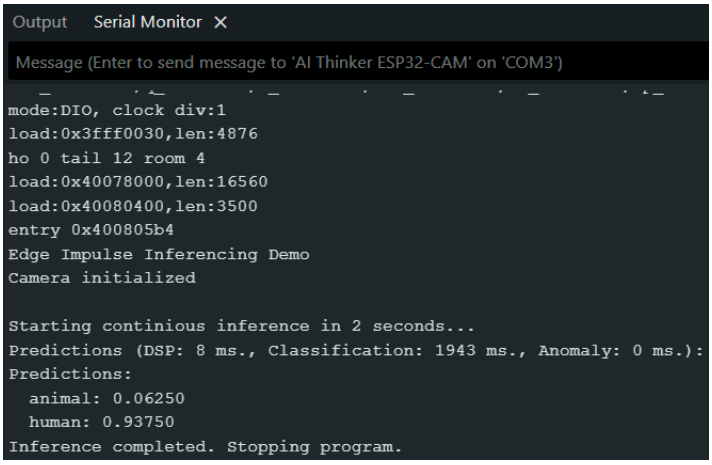


Fig:- 8 Output for an human image

---

13. Applications

This system can be applied in several real-world scenarios:

- Smart security systems
- Wildlife monitoring
- Smart farming
- Intrusion detection
- Automated surveillance systems

The low-cost nature of ESP32-CAM allows deployment of multiple devices in distributed environments.

---

14. Limitations

Despite the successful implementation, the system has certain limitations:

- Limited dataset diversity
- Performance affected by poor lighting conditions

- Limited processing power of ESP32
- Static image classification rather than continuous detection

Future improvements can address these limitations.

---

15. Future Work

Possible extensions of this project include:

- Real-time object detection
- Multi-class classification
- Integration with mobile notification systems
- Cloud synchronization
- IoT integration with alert systems

---

16. Conclusion

This project demonstrates the successful implementation of an **Edge-AI based image classification system using ESP32-CAM and Edge Impulse**.

The system performs **human versus animal classification directly on an embedded device**, eliminating the need for cloud processing.

The results show that lightweight machine learning models can be efficiently deployed on low-cost microcontrollers while maintaining high classification accuracy.

This project highlights the practical potential of **TinyML and Edge AI for embedded vision applications**.

---

17. References

1. https://www.kaggle.com/datasets/ashwingupta3012/human-faces

2. https://www.kaggle.com/datasets/likhon148/animal-data

3. https://docs.edgeimpulse.com/hardware/deployments/run-webassembly-browser